Technical Screening Assignment

Build an AI-Powered Natural Language to SQL System

Position: AI/ML Developer Intern | Round: 1 of 2 | Duration: 3–5 Days

Candidates who successfully complete this assignment will be invited for a technical interview.

Confidential — Do Not Distribute

About This Assignment

Thank you for your interest in the AI/ML Developer Intern position. As part of our selection process, we ask all shortlisted candidates to complete this take-home technical assignment before being invited to the interview round.

This assignment is designed to assess your ability to read technical documentation, integrate third-party AI tools, build a working backend API, and write clean, well-structured code — all skills you will use on the job.

What You Need to Build

Your task is to build a working Natural Language to SQL (NL2SQL) chatbot using Vanna AI 2.0 and FastAPI. The system should allow users to ask questions in plain English and get results from a database — without writing any SQL themselves.

Example

- User asks: "Show me the top 5 customers by total purchase amount"
- System generates SQL, executes it, and returns the results with a summary

How You Will Be Evaluated

Area	What We Look For
Functionality	Does the system run end-to-end without errors?
Code Quality	Is the code clean, readable, and well-organised?
Use of Vanna 2.0	Did you use the correct Vanna 2.0 APIs (not 0.x patterns)?
Documentation	Is the README clear enough for someone else to run the project?
Test Coverage	How many of the 20 test questions produce correct SQL?
Problem Solving	How did you handle edge cases, errors, and failures?

> *Note: There is no single correct solution. We are more interested in how you approach and solve the problem than in a perfect score. Candidates who submit a working project with honest documentation of failures will be preferred over those who submit nothing.*

What You Will Build

The system follows this pipeline:

```
User Question (English)
        |
        v
   FastAPI Backend
```

```
      |
      v
Vanna 2.0 Agent
(GeminiLlmService + RunSqlTool + DemoAgentMemory)
      |
      v
SQL Validation (SELECT only, no dangerous queries)
      |
      v
Database Execution (SQLite via built-in SqliteRunner)
      |
      v
Results + Summary + Chart returned to user
```

Tech Stack (Required)

Technology	Version	Purpose
Python	3.10+	Backend language
Vanna	2.0.x	AI Agent for NL2SQL
FastAPI	Latest	REST API framework
SQLite	Built-in	Database (no installation needed)
LLM Provider	Your choice (see below)	LLM for SQL generation — pick one free option
Plotly	Latest	Chart generation

Choose Your LLM Provider (Pick Any One)

Vanna 2.0 supports multiple LLM providers. You are free to use whichever of the following free options you prefer. Mention your choice clearly in your README.

Option	Provider	Model	Free Tier	SDK to Install	Vanna Import
A	Google Gemini	gemini-2.5-flash	Yes — via AI Studio	pip install google-genai	from vanna.integrations.google import GeminiLlmService
B	Groq	llama-3.3-70b-versatile	Yes — free tier available	pip install groq	from vanna.integrations.openai import OpenAILlmService (OpenAI-compatible)
C	Ollama (Local)	llama3 / mistral	Yes — fully free, runs locally	Install Ollama app	from vanna.integrations.openai import OpenAILlmService (base_url=localhost)

API Key links:

- Google Gemini: https://aistudio.google.com/apikey (free, sign in with Google)
- Groq: https://console.groq.com (free tier, sign up required)
- Ollama: https://ollama.com — no API key needed, runs on your machine

> *Note: ChromaDB is NOT required for Vanna 2.0. The old vn.train() pattern with ChromaDB belongs to Vanna 0.x. Vanna 2.0 uses an Agent Memory system (DemoAgentMemory) instead.*

Step 1: Create SQLite Database

Create a file called setup_database.py that creates a SQLite database with the following schema. This simulates a small clinic management system.

Table 1: patients

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment
first_name	TEXT NOT NULL	Patient first name
last_name	TEXT NOT NULL	Patient last name
email	TEXT	Patient email
phone	TEXT	Phone number
date_of_birth	DATE	Birth date
gender	TEXT	M / F
city	TEXT	City name
registered_date	DATE	When they registered

Table 2: doctors

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment
name	TEXT NOT NULL	Doctor full name
specialization	TEXT	e.g., Dermatology, Cardiology
department	TEXT	Department name
phone	TEXT	Contact number

Table 3: appointments

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment
patient_id	INTEGER	FK to patients.id
doctor_id	INTEGER	FK to doctors.id
appointment_date	DATETIME	When the appointment is
status	TEXT	Scheduled / Completed / Cancelled / No-Show
notes	TEXT	Optional notes

Table 4: treatments

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment
appointment_id	INTEGER	FK to appointments.id
treatment_name	TEXT	Name of procedure
cost	REAL	Treatment cost
duration_minutes	INTEGER	How long it took

Table 5: invoices

Column	Type	Description
id	INTEGER PRIMARY KEY	Auto-increment
patient_id	INTEGER	FK to patients.id
invoice_date	DATE	When invoice was created
total_amount	REAL	Total billed
paid_amount	REAL	Amount paid
status	TEXT	Paid / Pending / Overdue

Step 2: Insert Dummy Data

Write a script that inserts realistic dummy data into the database:

- 15 doctors across 5 specializations (Dermatology, Cardiology, Orthopedics, General, Pediatrics)
- 200 patients with realistic names, spread across 8–10 cities
- 500 appointments over the past 12 months, with varied statuses
- 350 treatments linked to completed appointments
- 300 invoices with a mix of Paid, Pending, and Overdue statuses

Requirements for Dummy Data

- Dates should be spread across the last 12 months (not all on the same date)
- Costs should be realistic (between 50 and 5000)
- Some patients should have many appointments (repeat visitors), some should have only 1
- Some doctors should have more appointments than others
- Include some NULL values in optional fields (email, phone, notes) to make it realistic

Deliverable: setup_database.py — creates schema + inserts all dummy data. Running it should produce a file clinic.db and print a summary:

```
"Created X patients, Y doctors, Z appointments..."
```

Step 3: Install Dependencies

Create a requirements.txt with all needed packages and install them.

Minimum Required Packages

```
vanna[sqlite]>=2.0.0          # base install
fastapi
uvicorn[standard]
plotly
pandas
python-dotenv
```

Then add the package for your chosen LLM provider:

If you chose	Add this to requirements.txt
Option A — Google Gemini	google-genai (and use vanna[gemini,sqlite])
Option B — Groq	groq
Option C — Ollama	No extra package needed — install Ollama app separately

> *Note: ChromaDB is not needed for Vanna 2.0. Specify your chosen LLM provider clearly in your README so reviewers can set it up correctly.*

Step 4: Initialize Vanna 2.0 Agent

Create a file called vanna_setup.py that sets up the full Vanna 2.0 Agent.

> *Note: Vanna 2.0 uses an Agent-based architecture. You do NOT write a custom SQL runner — Vanna provides a built-in SqliteRunner that handles the database connection for you.*

Build the following components:

1. Create an LLM Service using your chosen provider (GeminiLlmService, OpenAILlmService with Groq, or OpenAILlmService with Ollama)
2. Create a ToolRegistry and register: RunSqlTool, VisualizeDataTool, SaveQuestionToolArgsTool, SearchSavedCorrectToolUsesTool
3. Create a DemoAgentMemory instance (Vanna 2.0's learning system)
4. Create a UserResolver — a simple one that identifies all users as a default user
5. Create the Agent with all components connected

Correct Import Paths for Vanna 2.0

```
from vanna import Agent, AgentConfig
from vanna.core.registry import ToolRegistry
from vanna.core.user import UserResolver, User, RequestContext
from vanna.tools import RunSqlTool, VisualizeDataTool
from vanna.tools.agent_memory import SaveQuestionToolArgsTool,
SearchSavedCorrectToolUsesTool
from vanna.integrations.sqlite import SqliteRunner
from vanna.integrations.local.agent_memory import DemoAgentMemory
```

LLM Import — Use the one matching your chosen provider

```
# Option A — Google Gemini
from vanna.integrations.google import GeminiLlmService


# Option B — Groq (OpenAI-compatible)
from vanna.integrations.openai import OpenAILlmService
# Set base_url='https://api.groq.com/openai/v1' and your GROQ_API_KEY


# Option C — Ollama (local, no API key)
from vanna.integrations.openai import OpenAILlmService
# Set base_url='http://localhost:11434/v1' and api_key='ollama'
```

How you configure and wire these components is for you to figure out by reading the Vanna 2.0 documentation at https://vanna.ai/docs and the quickstart guide at https://vanna.ai/docs/tutorials/quickstart-5min.

Step 5: Seed Agent Memory with Example Q&A Pairs

Create a file called seed_memory.py.

> *Note: Vanna 2.0 does not use the old vn.train(ddl=..., sql=...) approach. Instead, Vanna 2.0 learns from successful interactions stored in DemoAgentMemory.*

Pre-seed the agent memory with at least 15 known good question–SQL pairs so the agent has a head start. Your 15 pairs must cover:

- Patient queries (count, list, filter by city/gender)
- Doctor queries (appointments per doctor, busiest doctor)
- Appointment queries (by status, by month, by doctor)
- Financial queries (revenue, unpaid invoices, average cost)
- Time-based queries (last 3 months, monthly trends)

Example Q&A Pairs

```
Q: "How many patients do we have?"
SQL: SELECT COUNT(*) AS total_patients FROM patients

Q: "Show revenue by doctor"
SQL: SELECT d.name, SUM(i.total_amount) AS total_revenue
     FROM invoices i
     JOIN appointments a ON a.patient_id = i.patient_id
     JOIN doctors d ON d.id = a.doctor_id
     GROUP BY d.name ORDER BY total_revenue DESC

Q: "Which city has the most patients?"
SQL: SELECT city, COUNT(*) AS patient_count FROM patients
     GROUP BY city ORDER BY patient_count DESC LIMIT 1
```

Step 6: Create the FastAPI Application

Create a FastAPI application (main.py). Vanna 2.0 provides a built-in VannaFastAPIServer that auto-generates production-ready endpoints.

Option A (Recommended) — Use VannaFastAPIServer

Vanna 2.0 ships with a built-in VannaFastAPIServer class that creates a production-ready app with a single call. This approach gives you a streaming chat endpoint and a ready-made web UI out of the box. Add your own /health route on top.

Option B — Custom Endpoints

Build custom endpoints that wrap the agent's send_message method directly for more control over the response format.

Required Endpoints

POST /chat

Request body:

```
{ "question": "Show me the top 5 patients by total spending" }
```

Response body:

```
{ "message": "Here are the top 5 patients by total spending...",
  "sql_query": "SELECT p.first_name, p.last_name,
SUM(i.total_amount)...",
  "columns": ["first_name", "last_name", "total_spending"],
  "rows": [["John", "Smith", 4500], ["Jane", "Doe", 3200]],
  "row_count": 5,
  "chart": { "data": [...], "layout": {...} },
  "chart_type": "bar" }
```

GET /health

```
{ "status": "ok", "database": "connected", "agent_memory_items": 15 }
```

Step 7: Add SQL Validation

Before executing ANY SQL generated by the AI, validate it:

6. Must be SELECT only — reject INSERT, UPDATE, DELETE, DROP, ALTER, EXEC
7. No dangerous keywords — reject: EXEC, xp_, sp_, GRANT, REVOKE, SHUTDOWN
8. No system tables — reject queries accessing sqlite_master or other system tables

If validation fails, return an error message — do NOT execute the query.

Step 8: Add Error Handling

- If the AI generates invalid SQL → return a friendly error message
- If the database query fails → catch the exception and return an error
- If no results are found → return a 'No data found' message

Step 9: Test with 20 Questions

Test your system with the following 20 questions and document the results:

#	Question	Expected Behavior
1	How many patients do we have?	Returns count
2	List all doctors and their specializations	Returns doctor list
3	Show me appointments for last month	Filters by date
4	Which doctor has the most appointments?	Aggregation + ordering
5	What is the total revenue?	SUM of invoice amounts
6	Show revenue by doctor	JOIN + GROUP BY
7	How many cancelled appointments last quarter?	Status filter + date
8	Top 5 patients by spending	JOIN + ORDER + LIMIT
9	Average treatment cost by specialization	Multi-table JOIN + AVG
10	Show monthly appointment count for the past 6 months	Date grouping
11	Which city has the most patients?	GROUP BY + COUNT
12	List patients who visited more than 3 times	HAVING clause
13	Show unpaid invoices	Status filter
14	What percentage of appointments are no-shows?	Percentage calculation
15	Show the busiest day of the week for appointments	Date function
16	Revenue trend by month	Time series
17	Average appointment duration by doctor	AVG + GROUP BY
18	List patients with overdue invoices	JOIN + filter
19	Compare revenue between departments	JOIN + GROUP BY
20	Show patient registration trend by month	Date grouping

Step 10: Document Your Results

Create a RESULTS.md file showing:

- For each question: the generated SQL, whether it was correct, and a result summary
- Count of how many out of 20 passed
- Any issues or failures, with an explanation of why they happened

Step 11: Write a README

Create a README.md that includes:

- Project description
- Setup instructions (step by step)
- How to run the memory seeding script
- How to start the API server
- API documentation with example requests/responses
- Architecture overview (brief)

Bonus Points (Optional)

These are NOT required but will earn extra credit:

9. Chart Generation — Successfully return Plotly charts for visualization queries
10. Input Validation — Validate that the question is not empty, not too long, etc.
11. Query Caching — Cache repeated questions to avoid redundant API calls
12. Rate Limiting — Add basic rate limiting to the /chat endpoint
13. Logging — Add structured logging for all steps

Submission Requirements

Please read the submission instructions carefully. Incomplete or incorrectly submitted assignments cannot be reviewed.

Files to Submit

```
project/
  setup_database.py      # Database creation + dummy data
  seed_memory.py         # Agent memory seeding with 15 Q&A pairs
  vanna_setup.py         # Vanna 2.0 Agent initialization
  main.py                # FastAPI application
  requirements.txt       # All dependencies
  README.md              # Setup & usage documentation
  RESULTS.md             # Test results for 20 questions
  clinic.db              # Generated database file
```

How to Submit

- Push to a public GitHub repository and share the link with us by email
- Include a clear commit history — we review your commits to understand your thought process
- Make sure the project runs with the command below:

```
pip install -r requirements.txt && python setup_database.py \
  && python seed_memory.py && uvicorn main:app --port 8000
```

Important Notes — Please Read

14. You MUST use Vanna 2.0 — not Vanna 0.x. The API is completely different. The old vn.train(), VannaBase, and ChromaDB pattern no longer apply. Read the 2.0 docs carefully before starting.
15. You must use one of the three approved free LLM providers: Google Gemini, Groq, or Ollama. Clearly state your choice in your README. Do not use paid APIs — we will not reimburse costs.
16. Do NOT hardcode any API key — use environment variables or a .env file. Example:

```
# .env file — for Gemini
GOOGLE_API_KEY=your-key-here


# .env file — for Groq
GROQ_API_KEY=your-key-here


# Ollama needs no API key — just run: ollama pull llama3
```

17. Use the correct import paths — Vanna 2.0 has a different package structure from 0.x. Wrong imports are a very common mistake.
18. SQLite is intentional — we want to see if you can build the pipeline. In production we use SQL Server, but SQLite keeps the assignment simple.
19. Reach out if you are stuck — email us at hiring@company.com. It is better to ask than to submit incomplete work.
20. Time management — do not spend all your time on one step. A working system with basic features is better than a perfect Step 1 with nothing else. We respect that you have other commitments.

Resources

Resource	URL
Vanna AI Documentation	https://vanna.ai/docs
Vanna 2.0 Quickstart	https://vanna.ai/docs/tutorials/quickstart-5min
Vanna GitHub	https://github.com/vanna-ai/vanna
FastAPI Documentation	https://fastapi.tiangolo.com
Google AI Studio (Gemini Free Key)	https://aistudio.google.com/apikey
Groq Console (Free Tier)	https://console.groq.com
Ollama (Free Local LLM)	https://ollama.com
Plotly Python	https://plotly.com/python

What Happens After You Submit

Stage	Timeline	Details
Assignment Review	3–5 business days after submission	Our team reviews your code, README, and test results
Shortlisting	Within 1 week	Candidates with working submissions are shortlisted for the interview
Technical Interview	Scheduled individually	A 45–60 min interview where you walk us through your code and answer follow-up questions
Final Decision	Within 1 week of interview	We will inform all candidates of the outcome

> *Note: During the interview, you will be asked to explain your design choices, walk through your code, and discuss how you would improve the system. Make sure you understand every part of what you submit.*

We wish you the best of luck! We look forward to reviewing your submission.